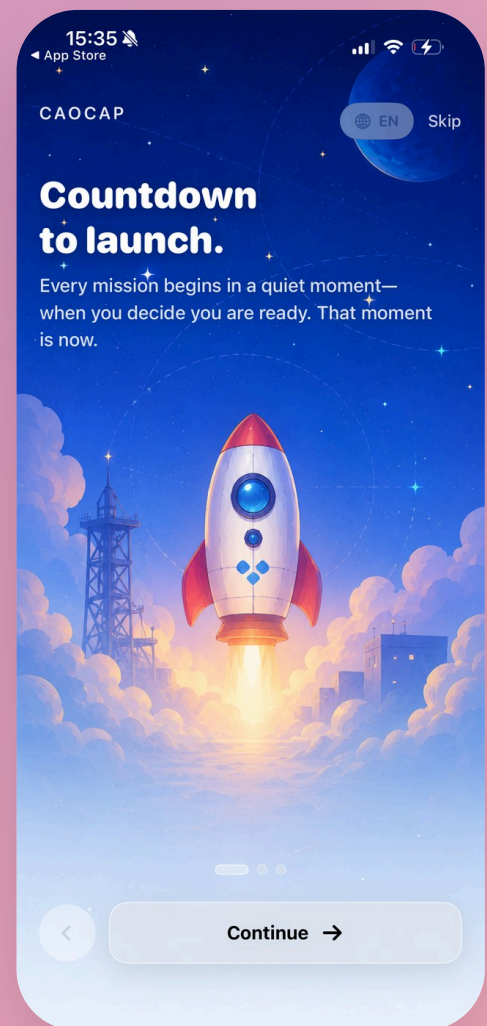


The first three minutes, *redesigned.*

A UX audit of the CAOCAP onboarding and first-run experience, with a screen-by-screen review and a restructured flow modeled on the best playful learning apps.



A good idea, held back by its *first mile*.

CAOCAP, audited.

CAOCAP's core idea is genuinely good: teach children real coding through play, guided by an AI co-pilot, inside their own games. The target and the pitch are clear, and that clarity is worth stating up front, because the opportunity here is real.

The onboarding falls short of that idea in two connected ways. It teaches the interface before it teaches the child anything, and a good part of the flow is spent on mood and narrative that does not shape what comes next or move the child toward a result. Duolingo, the reference you named, handles this differently: nearly every one of its onboarding screens either collects an answer that shapes what follows or hands over a small piece of the actual product. CAOCAP's early screens lean more on atmosphere than on payoff, and once the flow does reach the product, it introduces it through professional developer conventions, a node canvas, a command palette, a code diff, that ask more of a child than they can give at that point. Neither issue is hard to fix, but both are worth addressing together.

The central recommendation: restructure the first session around one activation moment, the child making a visible change to a running mini game within the first two to three minutes. Every screen either serves that moment or moves behind it. Account creation, subscription messaging, code diffs, and the canvas all move after the first win.

IN THIS AUDIT

Product understanding	03
The problem and why it matters	05
Screen-by-screen findings	07
Recommended changes	16
Proposed flow and measurement	19

What CAOCAP *is.*

CAOCAP is a mobile app-building and coding-education product for children and young beginners. The learning model is constructionist: children learn HTML, CSS, and JavaScript by building and modifying small apps and games, supported by an AI assistant. Games and apps are the means; the learning is the end. The stated inspiration is Duolingo, a mascot-led, gamified learning product with short session loops.

The core concepts I observed in the current build:

Canvas & *nodes*

Projects live as nodes on an infinite canvas. Nodes can be apps, sub-canvases, or system items: Profile, Settings, Pro Subscription, Activity, Daily challenges, Help.

CoCaptain / *CoStar*

A selectable AI co-pilot character that chats with the user, executes build requests, and explains decisions. Requests produce code changes that the user reviews and applies.

Command *palette*

A single input (marked with a ⌘ glyph) that acts as command launcher and AI prompt field.

Templates

Ready mini-apps (Pac-Man, XO/tic-tac-toe, Hello World) demonstrating what can be built.

Tutorial

A guided seven-step tooltip tour that walks the user from the canvas into a first AI-assisted edit of a Hello World app.

Fourteen screens, three *minutes.*

The screenshots I took on my iPhone show the following sequence for a brand-new user (English UI, session length roughly three minutes). Read as a funnel, the flow asks the child to absorb a brand narrative, an interface model (canvas, nodes, palette), and a professional review workflow before it delivers any moment of creation. The first tangible outcome of the session is a renamed text string, gated behind a code diff.

#	SCREEN	PURPOSE AS DESIGNED
1-3	Story slides (Countdown to launch / The view changes everything / One small step)	Emotional framing, brand world
4	Choose your co-pilot (CoCaptain / CoStar)	Character selection
5	What's your coding level? (slider, HTML/CSS/JS logos)	Skill self-assessment
6	Tap to launch your coding journey	Transition
7	Main canvas with ten nodes incl. Pro Subscription, Settings	Home, entry to Tutorial
8-10	Tutorial tooltips: command palette, Ask CoCaptain, send message	Teach primary input
11	CoCaptain chat, six-line greeting, edit task prompt	First AI interaction
12-13	Pending changes: code diff review, Apply / Reject	Confirm the edit
14	Celebration: Your first app is on the canvas, next step via Help	Reward, continuation

Signups happen. Sessions *end*.

The problem, and the research that shows why the first session is the highest-leverage place to fix it.

Children review a pull request before they *play.*

Signups happen, but conversion to active use and retention are low, and the feedback keeps repeating the same line: nice concept, bad UX. That combination, interest at the top of the funnel, but people leaving during the first session, is what points me at onboarding and first-run experience rather than at positioning or acquisition. First sessions decide retention. Roughly **77% of daily active users stop using an app within the first three days** of installing it, and average Day-1 retention is about 22.6% on Android and 25.6% on iOS (AppsFlyer benchmarks, via VWO, 2025). Onboarding itself is rarely finished: in Q2 2025, only about **8.4% of users completed onboarding within 30 days** across measured apps (scandiweb, 2026), which means most people never even reach the product's real value.

The guidance across sources agrees on timing: users should **feel the core value within the first 60 seconds**, and every screen placed before that moment adds to the drop-off (lowcode.agency, 2026). This is why onboarding is one of the highest-leverage places to fix a product: good onboarding can **lift retention by up to 50%**, and single, targeted changes, like a delayed signup or a personalization question, have moved conversion by double digits in published A/B tests (scandiweb 2026; Adapty 2025). For a children's product the bar is even higher: few and large tap targets, three to four per screen at most, copy written to the reading level of the target age, immediate visual and audio feedback for every action, and no reliance on abstract symbols or conventions borrowed from professional software (UX Magazine; Ungrammary; Aufait UX).

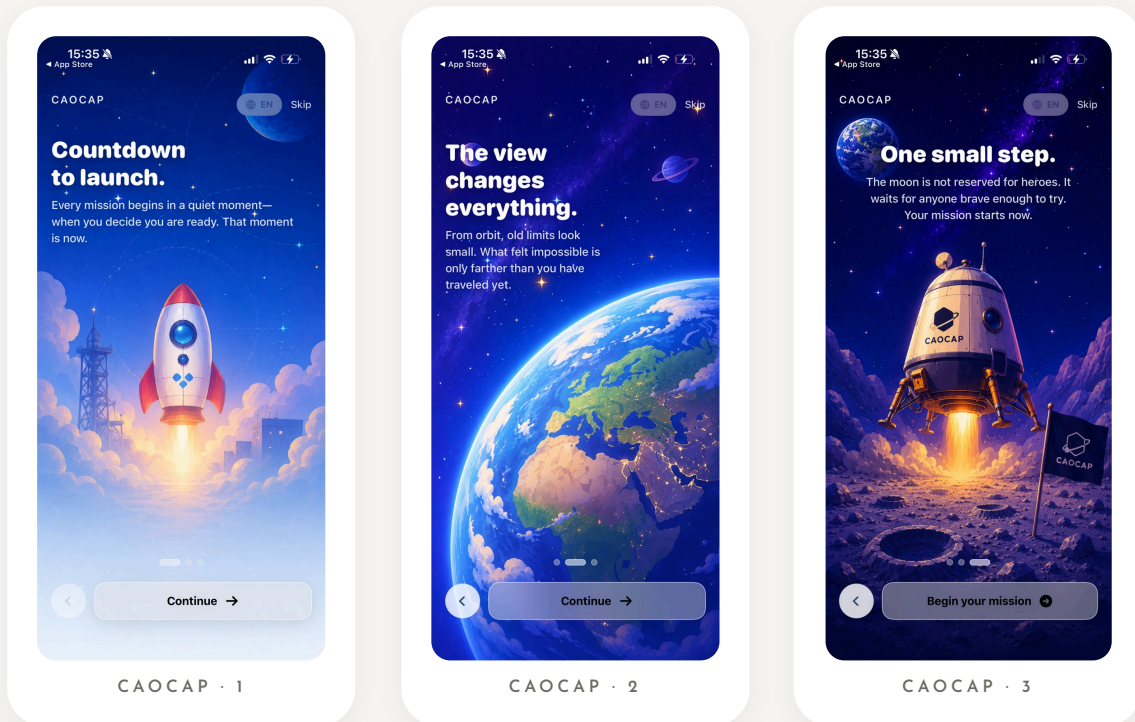
Put simply, CAOCAP currently front-loads narrative, interface concepts, and developer workflow, and back-loads the value moment; children are asked to review a pull request before they are even allowed to play. The redesign needs to invert that order.

Screen by screen, what actually *happens.*

Eight findings across the fourteen observed screens, each documented with the screen itself and, where useful, the Duolingo pattern beside it.

FINDING 01 · SCREENS 1-3

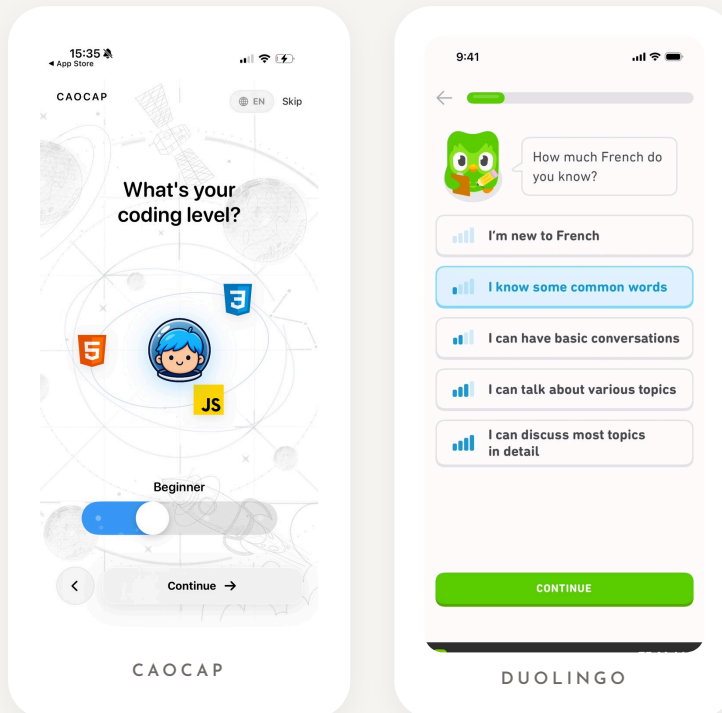
Three static story slides before *any interaction*.



The flow opens with three consecutive full-screen slides carrying adult, metaphor-heavy copy ("The moon is not reserved for heroes. It waits for anyone brave enough to try."). No slide collects input, adapts to the user, or shows the product. Nielsen Norman Group's mobile onboarding research warns against exactly this pattern: users want to do something, not watch something, and long upfront instruction or narrative gets skipped or abandoned. The reading level is also well above the comprehension of younger target users. Duolingo solves the same emotional job differently: its mascot appears once with a single line, and all further narrative is delivered as reactions to the user's own answers, so storytelling and data collection happen on the same screens.

FINDING 02 · SCREEN 5

A skill question asked in *file formats*.

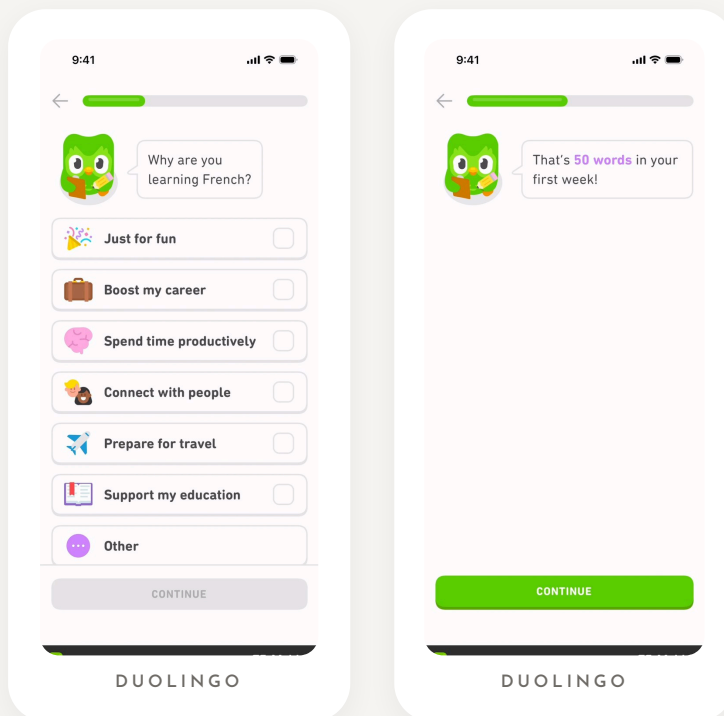


A child cannot self-assess against file-format logos; HTML5, CSS3, and JS shields do not communicate anything to a child, most will not know what these symbols even mean. The continuous slider is also the wrong control for what is a discrete choice. Duolingo phrases the same question as feelings in first person ("I'm new to French", "I know some common words") with small bar icons indicating progression.

Beyond the control itself, this is the only personalization question in the whole flow, and its answer visibly changes nothing afterwards. The app never routes a beginner and an experienced child down different paths, never adjusts the difficulty of the first project, and never changes what CoCaptain says next, so the question is asked without a purpose. If an answer does not change anything downstream, it should not be asked at all. Research on onboarding personalization shows this is a significant missed lever: in Adapt's published A/B test, adding upfront personalization questions plus a "customizing your experience" moment lifted trial starts by 8.5% and paying conversions by 17%.

FINDING 03 · FLOW-LEVEL GAP

No question that shapes the *first project*.



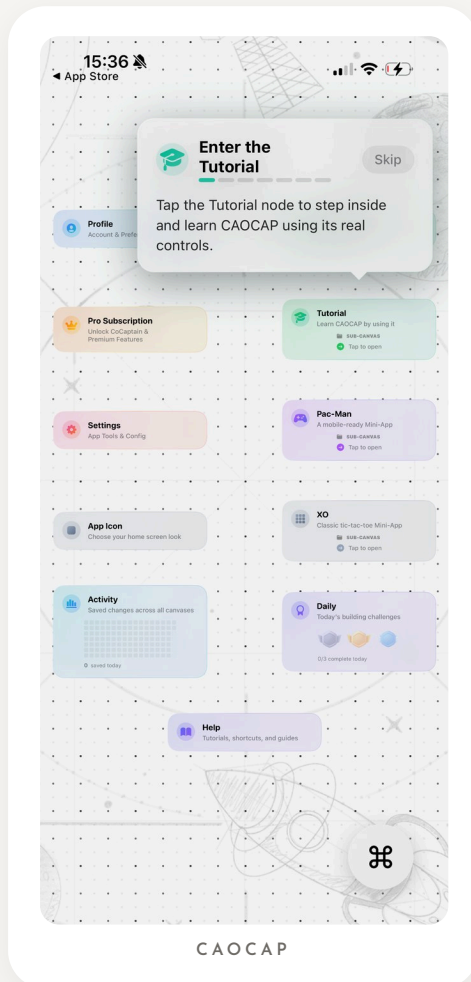
Duolingo's motivation question, and the immediate payoff projection on the very next screen: "That's 50 words in your first week!"

Duolingo's onboarding asks for language, motivation, level, and daily goal, then immediately converts the commitment into a concrete outcome. Answers visibly shape the experience and are converted into a promise.

CAOCAP never asks the one question that matters most for a builder product: what do you want to build? The first session therefore cannot feel personal, and the child's own intent is never activated. This is the single most important missing screen.

FINDING 04 · SCREEN 7

First landing: a ten-node *canvas*.



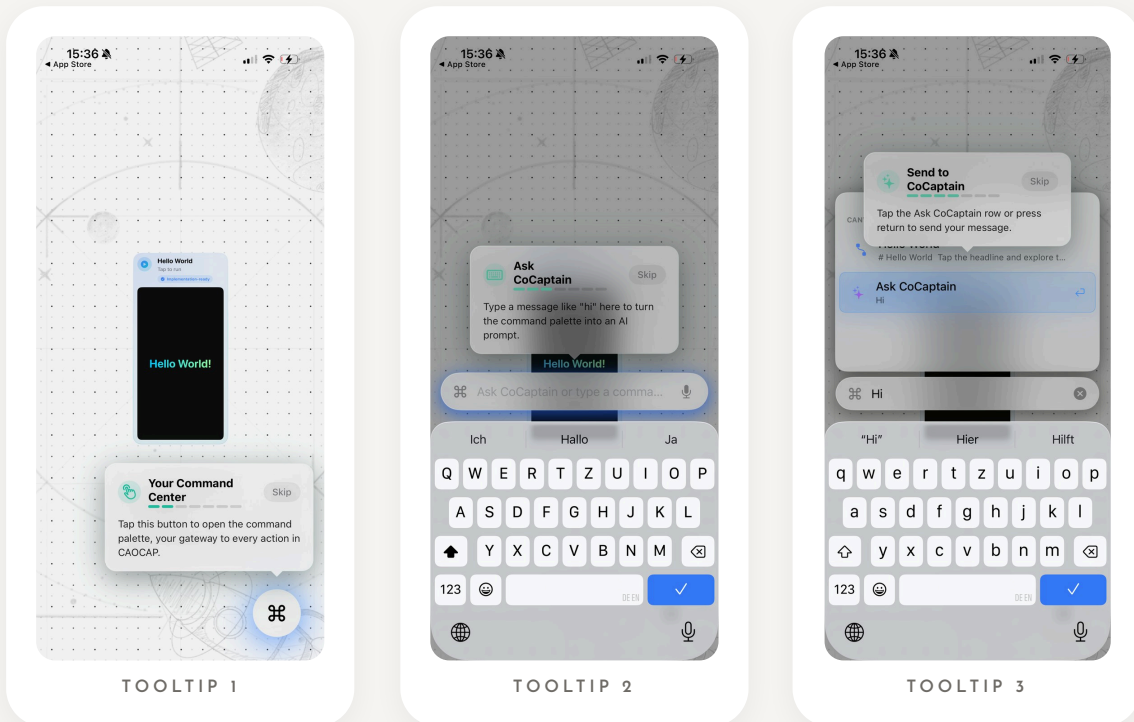
The first-run canvas: Profile, Pro Subscription, Settings, App Icon, Activity, Tutorial, Pac-Man, XO, Daily, Help.

A brand-new child user lands on an infinite canvas with ten unexplained objects. Kids-UX guidance recommends three to four large targets per screen for young users; ten mixed-purpose nodes is a cognitive dump, and the actual next step (Tutorial) competes with nine distractors, every one of them a live tap target a child can hit by accident before they have understood what any of it does.

The canvas is, in effect, a file browser, and no product's aha moment is its file browser.

FINDING 05 · SCREENS 8-10

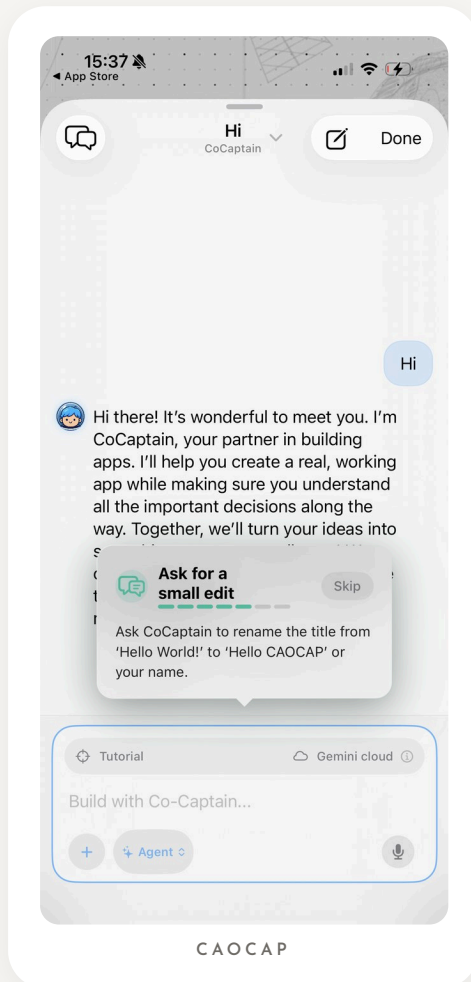
A command palette as the child's *front door*.



The single, do-everything input, an omni button that launches commands and takes AI prompts from one place, is a genuinely good idea for a builder app; it is the execution that breaks it. The ⌘ glyph representing that input is a Mac keyboard symbol with no meaning to a child on a phone. The control then shape-shifts from command launcher to AI prompt field, two mental models in one element, and the tutorial needs three instruction bubbles just to get one message sent. When an input requires three tooltips, the input is the problem. The chat with the co-pilot should simply be the interface in beginner mode, carrying the omni-button idea without the keyboard-shortcut framing.

FINDING 06 · SCREEN 11

CoCaptain opens with a *text wall*.



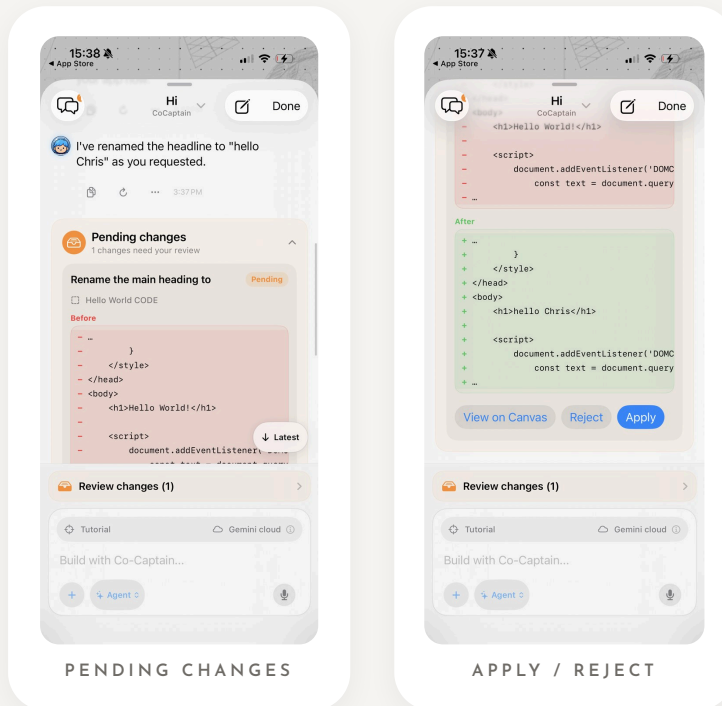
Children's reading comprehension develops gradually; kids-UX guidance calls for short sentences, one idea per message, and ideally voiceover. The greeting also surfaces implementation details (model selector "Gemini cloud", "Agent" toggle) that belong behind an advanced mode, not in a child's first conversation.

Duolingo's equivalent moment is one line ("Hi there! I'm Duo!") followed by "Just 7 quick questions before we start your first lesson!": expectation set, zero lecture.

Six lines about process and "important decisions", plus visible system labels ("Agent", "Gemini cloud").

FINDING 07 · SCREENS 12-13

A git-style diff gates the first *creative act*.



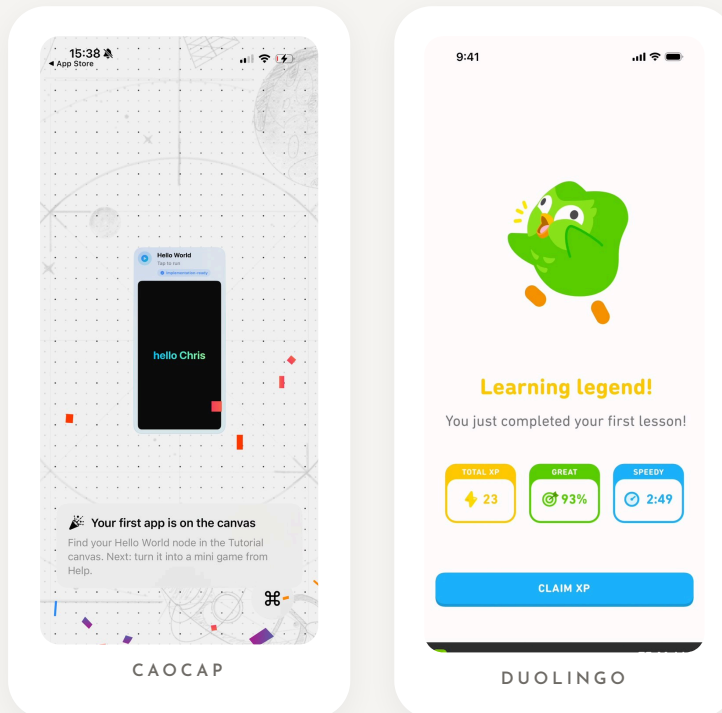
The child asks to rename a headline and receives red/green code blocks with Apply, Reject and View on Canvas.

This is the moment that produces "nice concept, bad UX" reviews. A rename request returns a professional code-review workflow: truncated diff lines, before/after blocks, and a three-button decision. The effort the child invests goes into bureaucracy rather than creation, which inverts the endowed-progress effect that makes Duolingo's first lesson work (effort invested into visible progress makes abandoning feel like a loss).

In a first session, changes should apply instantly to a live preview with a prominent Undo; reading diffs can become a deliberate lesson for advanced levels later. The panel also contains a grammar defect on a flagship screen ("1 changes need your review").

FINDING 08 · SCREEN 14

A celebration that exits into *documentation*.



CAOCAP rewards a renamed string and points to Help. Duolingo rewards with XP, accuracy and time, then chains into the next commitment.

The confetti instinct is right, but the celebrated achievement is a renamed heading, and the suggested continuation is "turn it into a mini game from Help". Sending a first-session child to documentation is where the session ends.

Duolingo closes its first lesson with a stats screen (XP, accuracy, time), an immediate claimable reward, and a direct chain into the next commitment (streak goal, then profile). The reward must match a real creation, and the next step must be one tap, not a documentation lookup.

Every screen earns its place, or *moves*.

Seven changes, each backed by evidence from Duolingo and other playful learning apps. The guiding metric: the child makes a visible change to a running mini game within the first two to three minutes.

R1 Replace the story slides with a reacting *mascot*

Cut the three slides to one character introduction with a single line ("Hi! I'm CoCaptain. Let's build your first game today.") plus one trust line for the skeptical parent or child ("No experience needed. You'll have a real game in 3 minutes."). Deliver all further narrative as CoCaptain's short reactions to the user's own answers, exactly as Duo does. This preserves the emotional work of the slides while removing three passive screens, and it is the pattern behind Duolingo's documented onboarding performance.

R2 Ask questions that shape the product, kid-worded, one tap each

Five screens, one tap each, progress bar on top, CoCaptain reacting between answers. Adapty's A/B test of personalization questions plus a tailoring moment produced +8.5% trial starts, +17% paying conversions, and +22% ARPU against a generic flow. The questions are not friction; unpersonalized screens are. The five questions follow on the next page.

R3 Land in a running game, not on the canvas

After the questions, open the chosen template already running, full screen. CoCaptain: "This is your game. Let's make it yours." The canvas is not shown in the first session at all; it becomes the home the child returns to afterwards, and in beginner mode it contains at most the child's game, Daily, and Tutorial. Pro Subscription moves behind a parent area entirely (see R6). This implements the value-within-60-seconds rule and respects the three-to-four-targets guidance for children.

THE FIVE QUESTIONS

- 01 What do you want to build first?**

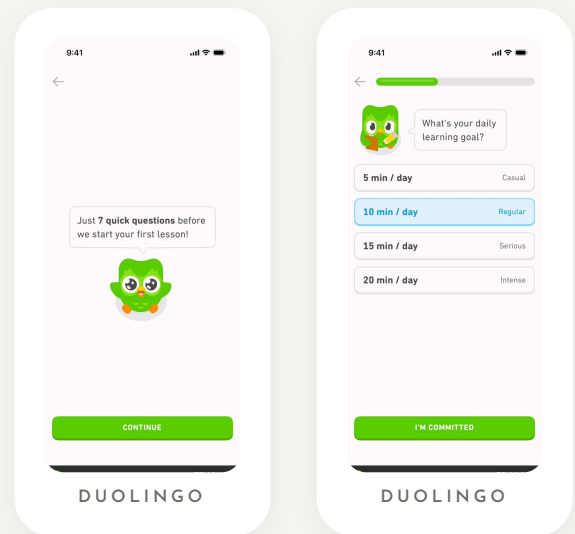
Picture cards: jump game, quiz, virtual pet, drawing toy. The answer selects the first project template. This replaces nothing in the current flow; it is new, and it is the most important screen in the redesign.
- 02 Have you built anything with code before?**

Kid-worded levels with bar icons: "First time!", "A little (Scratch, Roblox)", "I already write code". Replaces the slider and the technology logos. Advanced answers route past hand-holding, which also addresses the "too babyish" risk for 11-to-12-year-olds, mirroring Duolingo's placement-test fork.
- 03 How did you find CAOCAP?**

Attribution, asked early while the memory is fresh. Duolingo places this question second in its own flow; the answer quality pays for the screen.
- 04 One outcome-preview screen.**

Three icons: build real games, share with friends, learn real code. Mirrors Duolingo's "Here's what you can achieve in 3 months!".
- 05 How much building time per day?**

Followed immediately by a payoff projection in CoCaptain's voice: "Nice! In your first week you'll build 3 mini games." Duolingo converts the same commitment into "That's 50 words in your first week!" on the very next screen.



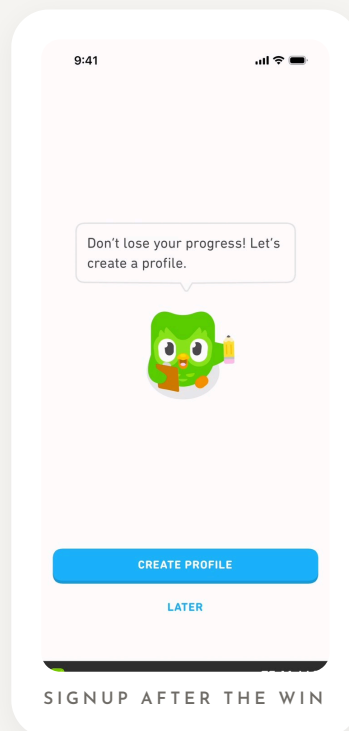
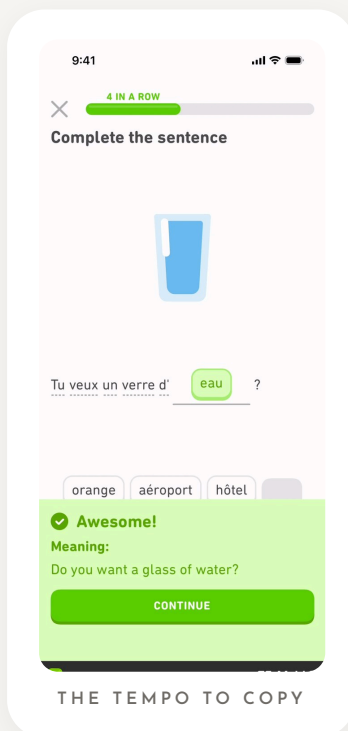
Duolingo's question cadence: expectation setting up front, then one question per screen with a progress bar and a reacting mascot.

R4 First build as a lesson: micro-tasks, instant apply, celebration

Structure the first build as three to four micro-tasks in Duolingo's lesson rhythm, progress bar on top, XP per task, each producing an instantly visible change in the running game: pick your character's color, make it jump higher, name your game, add a sound. Changes apply live with a prominent Undo. The diff review disappears from beginner mode; reading and approving code becomes a deliberate, later lesson (it is a genuinely good learning device in the right place). Close with a stats screen in the Duolingo pattern: XP earned, "You changed 4 things", time, confetti, and the game playable full screen. That screen, not a renamed heading, is the aha moment.

R5 Move account creation after the win, framed as saving the creation

Duolingo's team has documented that moving signup behind the first lesson increased daily active users by about 20% ("People thought we were crazy... but the data was unambiguous", via First Round Review and Taplytics). The psychological mechanism is loss aversion on endowed progress. CAOCAP's equivalent: "Save your game so it's here tomorrow!". The commitment ladder then follows Duolingo's proven order, each ask placed directly after a win: builder streak goal ("build something 7 days in a row"), notification permission asked by CoCaptain with a stated reason, home-screen widget last.



Left: Duolingo delivers a green success bar with sound roughly every ten seconds. Right: the profile ask placed after the first lesson and framed as protecting earned progress: "Don't lose your progress! Let's create a profile."

R6 Separate the parent track from the child track

Age is collected first in the profile step (as Duolingo does) and routes to a parent gate: account, payment, permissions, and language are handled once by the parent and saved, so the device can be handed back to the child without further interruptions (UX Planet's guidance for kids' products). The child never sees a subscription card; upgrade messaging lives in the parent area and, as in Duolingo's "Awesome! You can upgrade anytime" screen, is never a blocker in the first session.

R7 Fix the seams

Grammar: "1 changes need your review" on the review panel. Vocabulary audit for beginner mode: "Implementation-ready", "Sub-canvas", "node", "Agent", "Gemini cloud" are internal or professional terms and should be hidden or renamed in kid-facing surfaces. Replace the ☒ glyph everywhere in kid-facing UI; the chat entry point carries CoCaptain's face instead.

Proposed flow at a glance

PHASE	SCREENS	CONTENT
1•Meet & personalize 60–90 s	6–7	CoCaptain intro (1 line) · trust line · what do you want to build? · experience level · attribution · outcome preview · daily goal + payoff projection
2•First build 2–3 min	5–6	Chosen template opens running · 3–4 micro-tasks with instant visible changes, XP, progress bar · stats + confetti + playable game
3•Commit & save	4–5	Save your game (profile, age first, parent gate) · builder streak goal · notifications with reason · widget

Measure the *cliffs.*

Before or alongside the redesign, instrument the funnel so every future change is testable. Primary KPI: percentage of installs reaching **first_change_applied** within the first session, and Day-1 return rate. My hypothesis from this audit is that the current cliffs sit at the canvas landing (F4) and the diff review (F7); the events below will confirm and quantify this before further investment.

EVENT	DEFINITION
onboarding_started	First screen rendered
template_chosen	Answer to "what do you want to build"
questions_completed	Last personalization answer given
first_change_applied	Activation: first visible change in the running game
first_build_completed	Stats screen reached
profile_created	Account saved (post-win)
streak_committed · notifications_allowed · widget_added	Commitment ladder steps
d1_return · d7_return	Retention checkpoints

What this audit *rests* *on.*

Benchmarks, published A/B tests, and kids-UX guidance referenced throughout the document.

Duolingo first-run flow, iOS (screenshots via Mobbin, 2026): onboarding, first lesson, learning goal, profile creation, widget.

VWO, The Ultimate Mobile App Onboarding Guide (2025): 77% of DAUs stop within three days; Day-1 retention 22.6% Android / 25.6% iOS.

scandiweb, User Onboarding Best Practices (2026): 8.4% onboarding completion within 30 days (Q2 2025); retention lift up to 50%; activation-metric discipline.

lowcode.agency, Mobile App Onboarding Best Practices (2026): value within 60 seconds; friction per added screen; 2–3 personalization questions.

Adapty, How to Fix Your Onboarding Flow (2025): personalization A/B test, +8.5% trial starts, +17% paying conversions, +22% ARPU.

Taplytics / First Round Review: Duolingo delayed-signup experiments, ~20% DAU increase from moving signup

behind the first lesson.

Nielsen Norman Group (mobile onboarding research, via Vidico 2026): users want to do something, not watch something; long upfront instruction gets skipped.

UX Magazine, Designing Apps for Kids: three to four large tap targets; avoid settings exposure for children.

Ungrammary, UX Design Tips for Children's Apps (2024): reading-level-appropriate copy, large type, voiceover support.

Aufait UX, UI/UX Design for Children (2024): immediate visual/auditory/haptic feedback over subtle cues; simplicity over complexity.

UX Planet, Designing Apps for Kids (2020): parent decisions saved during onboarding so the device can be handed to the child.

Next: the new flow, on *screens*.

The natural next step is turning this audit into the redesigned flow itself: wireframes for the three phases, copy for CoCaptain, and a clickable prototype to validate with real kids before development.